ErrorBoundary コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

ErrorBoundaryは、子コンポーネントで発生したエラーをキャッチし、エラー情報を表示するReactエラーバウンダリコンポーネントです。アプリケーション全体がクラッシュするのを防ぎ、エラー詳細を開発者に提供します。

1.2 使用場面

- ビューアコンポーネント（PetriNetViewer、ImageViewerなど）のラップ
- 複雑なUIコンポーネントのエラーハンドリング
- 開発・デバッグ時のエラー情報表示

1.3 アクセス権限

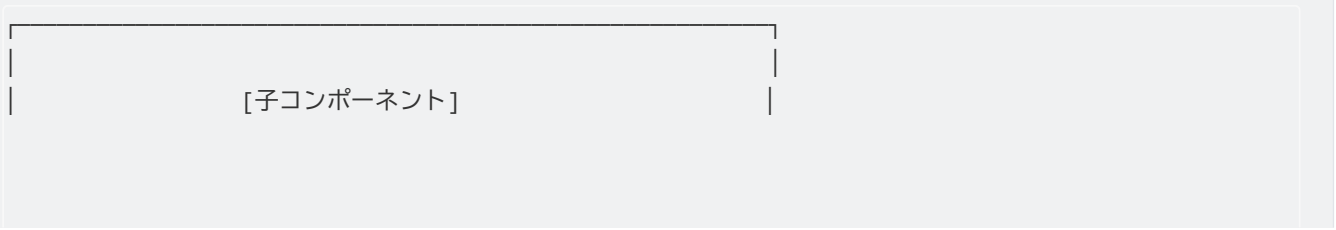
- **必須ロール**: なし（すべてのユーザー）
- **認証**: 不要（エラーハンドリング用コンポーネント）

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常時（エラーなし）





2.1.2 エラー発生時

2.2 主要UI要素

2.2.1 エラーアラート

- **メッセージ**: "コンポーネント表示エラー"
- **説明**: "このコンポーネントの表示中にエラーが発生しました。"
- **タイプ**: error
- **アイコン**: WarningOutlined

2.2.2 エラー詳細カード

- **タイトル**: "エラーの詳細"
- **サイズ**: small

- **Extra**: 再表示ボタン

2.2.3 エラーメッセージ

- **ラベル**: "エラーメッセージ:"
- **表示**: Paragraph、code形式、コピー可能
- **内容**: error.toString()

2.2.4 コンポーネントスタック

- **ラベル**: "コンポーネントスタック:"
- **表示**: Paragraph、code形式、コピー可能、スクロール可能
- **最大高さ**: 300px
- **内容**: errorInfo.componentStack

2.2.5 再表示ボタン

- **タイプ**: Primary
- **サイズ**: small
- **アイコン**: ReloadOutlined
- **ラベル**: "再表示を試みる"
- **機能**: エラー状態をリセットして子コンポーネントを再レンダリング

---

3. 機能仕様

3.1 エラーキャッチ機能

3.1.1 エラーの検出

```
static getDerivedStateFromError(error: Error): Partial<ErrorBoundaryState> {
  return { hasError: true };
```

```
}
```

- **React標準メソッド**: `getDerivedStateFromError`
- **実行タイミング**: 子コンポーネントでエラーが発生した直後
- **処理内容**: `hasError` フラグをtrueに設定

3.1.2 エラー情報の保存

```
componentDidCatch(error: Error, errorInfo: ErrorInfo): void {
  this.setState({
    error,
    errorInfo,
  });

  // エラーログ出力
  console.error('コンポーネントエラー:', error, errorInfo);
}
```

- **React標準メソッド**: `componentDidCatch`
- **実行タイミング**: エラー検出後
- **処理内容**:
  - errorとerrorInfoをStateに保存
  - コンソールにエラーログ出力

3.2 エラー状態のリセット

3.2.1 処理フロー

```
handleReset = (): void => {
  this.setState({
    hasError: false,
    error: null,
    errorInfo: null,
```

```
  });
};
```

- **機能**: エラー状態をクリアして通常表示に戻す
- **トリガー**: "再表示を試みる"ボタンクリック
- **効果**: 子コンポーネントが再マウントされる

3.3 条件分岐レンダリング

3.3.1 通常時

```
if (this.state.hasError) {
  // エラー表示
} else {
  return this.props.children;  // 子コンポーネントをそのまま表示
}
```

- `hasError === false` の場合、子コンポーネントを通常通り表示

3.3.2 エラー時

```
return (
  <div style={{ padding: 24 }}>
    <Space orientation="vertical" size="large" style={{ width: '100%' }}>
      <Alert ... />
      <Card ... />
    </Space>
  </div>
);
```

- `hasError === true` の場合、エラーUI表示

---

4. データ仕様

4.1 Props定義

```typescript
interface ErrorBoundaryProps {
  children: ReactNode;
}
```

Props説明:

Prop名	型	必須	説明
children	ReactNode	✅	ラップする子コンポーネント

4.2 State定義

```typescript
interface ErrorBoundaryState {
  hasError: boolean;
  error: Error | null;
  errorInfo: ErrorInfo | null;
}
```

State説明:

State名	型	初期値	説明
hasError	boolean	false	エラー発生フラグ
error	Error | null	null	エラーオブジェクト
errorInfo	ErrorInfo | null	null	Reactエラー情報（コンポーネントスタック含む）

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
-	（なし）	-	-	-	API呼び出しなし

備考:

- ErrorBoundaryはローカルのエラーハンドリングのみ
- 将来的にエラーログをバックエンドに送信する可能性あり

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
子コンポーネントで実行時エラー	エラーUIを表示、エラー詳細を提供
再表示ボタンクリック後もエラー	再度エラーUIを表示（ユーザーは問題が解決していないことを確認）

6.2 エラー処理コード

6.2.1 コンソールログ出力

```
console.error('コンポーネントエラー:', error, errorInfo);
```

- **目的**: 開発者がブラウザコンソールでエラーを確認できるようにする

6.2.2 エラーメッセージの表示

```
{error && (
  <div>
    <Text strong>エラーメッセージ:</Text>
    <Paragraph code copyable style={{ marginTop: 8 }}>
      {error.toString()}
    </Paragraph>
  </div>
)}
```

- **コピー機能**: ユーザーがエラーメッセージをコピーして報告できる

6.2.3 コンポーネントスタックの表示

```
{errorInfo && errorInfo.componentStack && (
  <div>
    <Text strong>コンポーネントスタック:</Text>
    <Paragraph
      code
      copyable
      style={{
        marginTop: 8,
        maxHeight: 300,
        overflow: 'auto',
        whiteSpace: 'pre-wrap'
      }}
    >
      {errorInfo.componentStack}
    </Paragraph>
  </div>
)}
```

- **スクロール**: 長いスタックトレースも表示可能

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/ErrorBoundary.tsx`

7.2 関連ファイル

- なし

7.3 依存コンポーネント

- なし（Ant Designコンポーネントのみ使用）

7.4 依存ライブラリ

- `react` : Component, ReactNode, ErrorInfo
- `antd` : Alert, Button, Space, Typography, Card
- `@ant-design/icons` : WarningOutlined, ReloadOutlined

---

8. 使用例

8.1 基本的な使用方法

```
import ErrorBoundary from '@/components/viewer/ErrorBoundary';
import PetriNetViewer from '@/components/viewer/PetriNetViewer';

function App() {
  return (
    <ErrorBoundary>
      <PetriNetViewer maimlData={maimlData} />
    </ErrorBoundary>
  );
}
```

8.2 複数コンポーネントのラップ

```
<ErrorBoundary>
  <div>
    <PetriNetViewer maimlData={maimlData} />
    <ImageViewer fileKey={fileKey} fileName={fileName} />
  </div>
</ErrorBoundary>
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline